

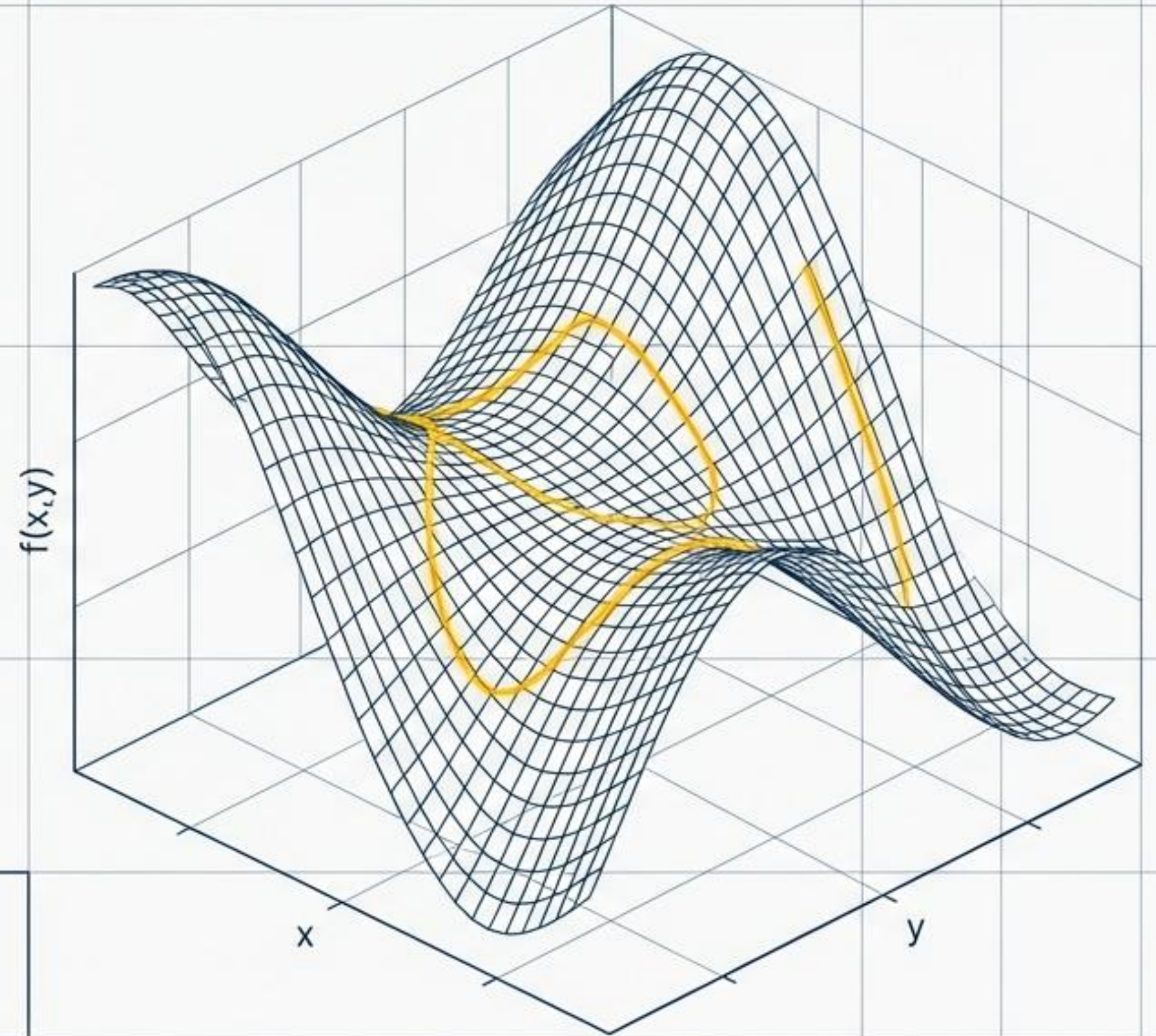
Ecuaciones No Lineales

Métodos Numéricos para la Localización de Raíces en Sistemas Dinámicos



Curso: Métodos Numéricos

Docente: Dr. Hermes Pantoja Carhuavilca

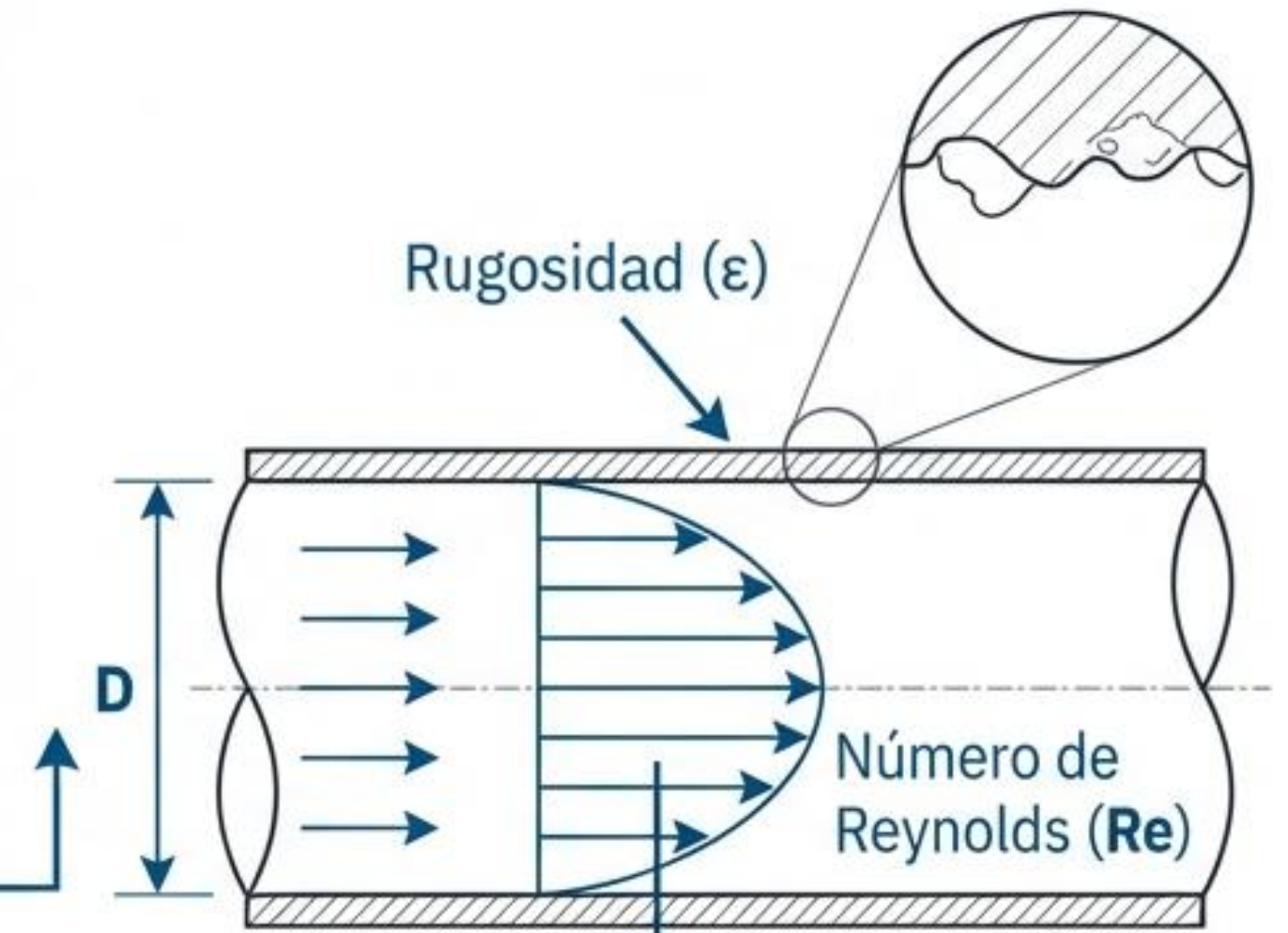


El Desafío: Fricción en Tuberías

En ingeniería de fluidos, determinar la caída de presión en una tubería es crucial. Esto requiere calcular el factor de fricción f , que se rige por la ecuación implícita de Colebrook para flujo turbulento:

$$\frac{1}{\sqrt{f}} = -2.0 \log_{10} \left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{\text{Re}\sqrt{f}} \right)$$

Punto Clave: Esta ecuación es **no lineal** e **implícita**: la incógnita f aparece en ambos lados y no puede ser despejada con álgebra simple. ¿Cómo resolvemos este problema fundamental?

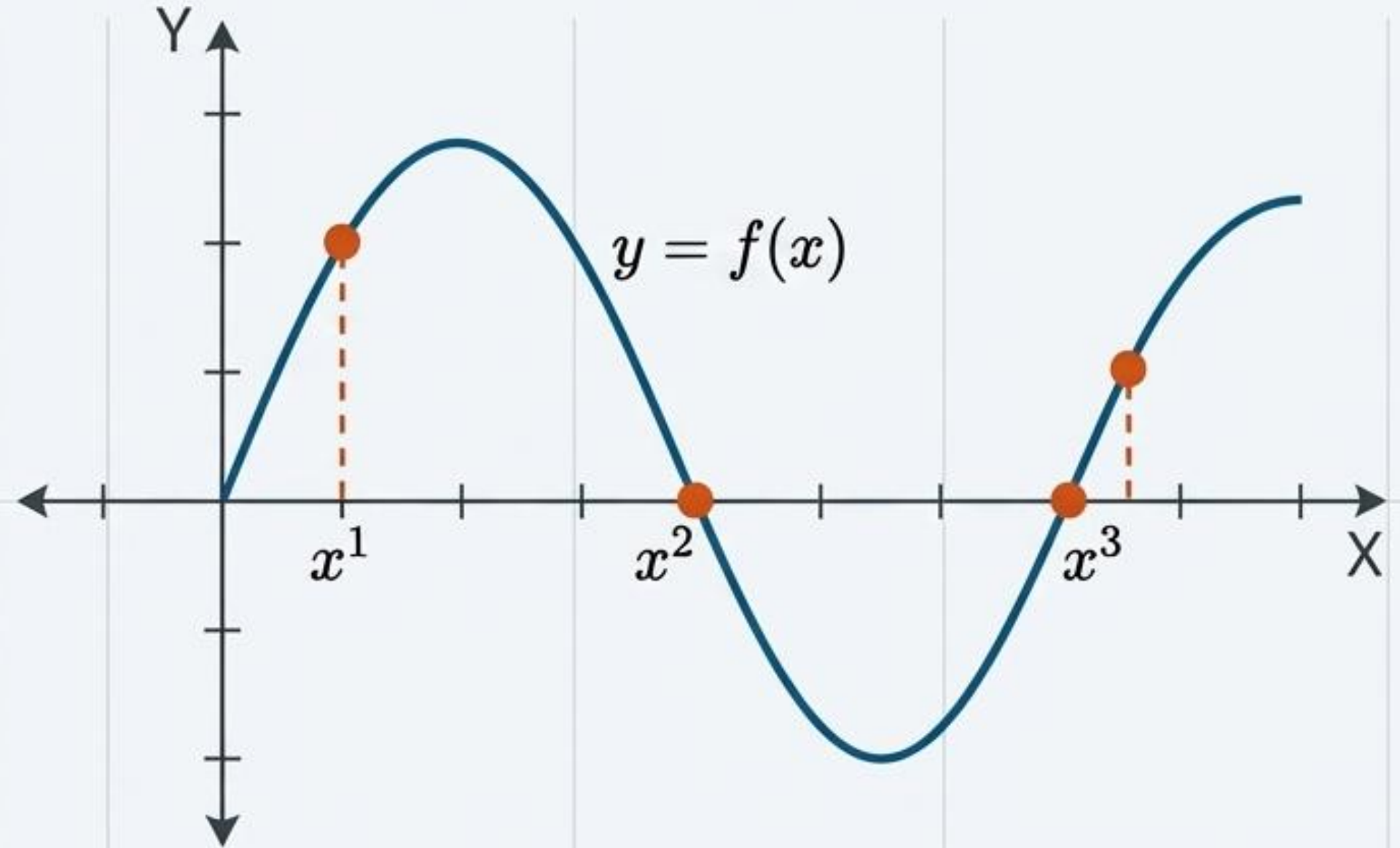


Fundamentos: ¿Qué Buscamos Exactamente?

Concepto: Buscamos la **raíz** (o el **cero**) de una función. Una raíz, x^* , es un valor de la variable independiente x para el cual la función $f(x)$ es igual a cero.

$$f(x) = 0$$

Interpretación Gráfica: Geométricamente, las raíces son los puntos donde la gráfica de la función $y = f(x)$ cruza el eje X.



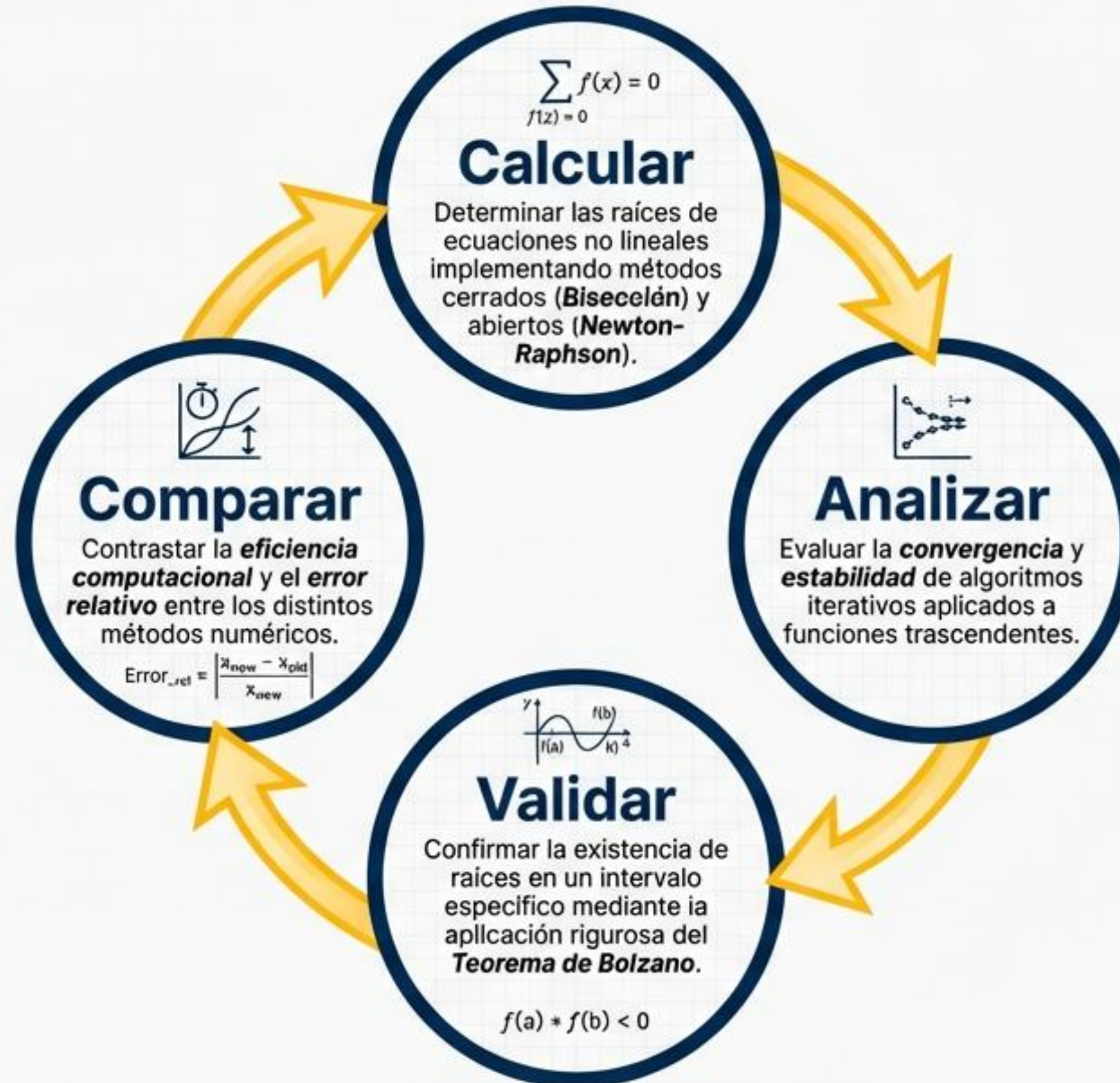
Reformulación del Problema:

Nuestro problema de fricción se puede reescribir como la búsqueda de la raíz de la función:

$$F(f) = \frac{1}{\sqrt{f}} + 2.0 \log_{10} \left(\frac{\epsilon/D}{3.7} + \frac{2.51}{\text{Re}\sqrt{f}} \right) = 0$$

Objetivos de la Sesión

Al finalizar la sesión, el estudiante habrá adquirido las competencias para:



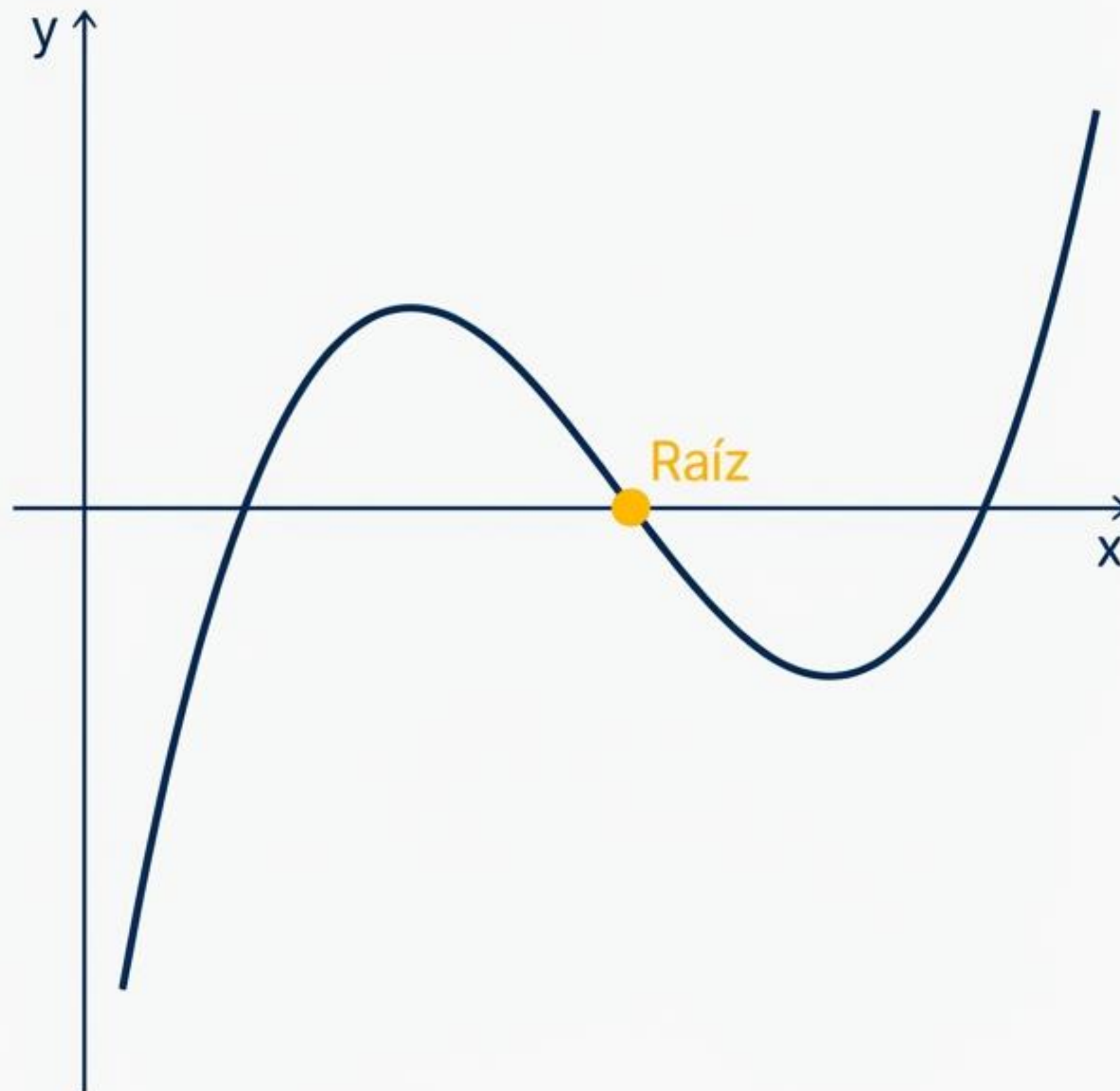
Definición del Problema

El objetivo fundamental es hallar un valor x tal que se cumpla la condición de nulidad:

$$f(x) = 0$$

A diferencia de las ecuaciones lineales, las funciones trascendentes o polinómicas de alto grado no siempre permiten un despeje analítico directo.

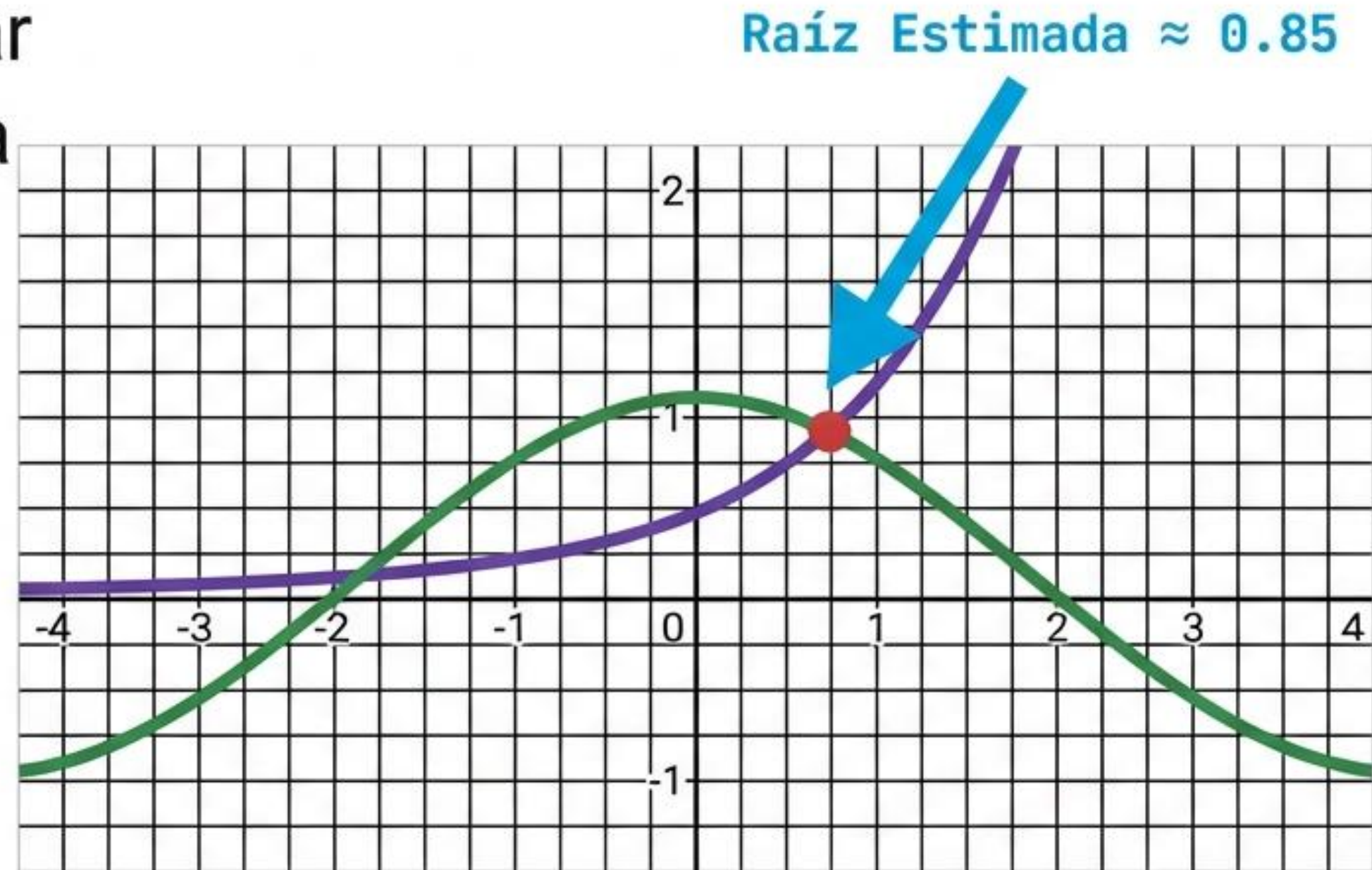
La Solución: Se utilizan métodos iterativos que refinan una aproximación inicial hasta que el error relativo aproximado ($|\varepsilon_a|$) es menor que una tolerancia predefinida.



A. Localización de Raíces: Método Gráfico

El primer paso: Inspección Visual

- **Técnica:** Graficar $f(x)$ o separar en $g(x)=h(x)$ para encontrar la intersección.
- **Ventajas:**  Intuición visual para seleccionar x_0 o el intervalo $[a, b]$.
- **Desventajas:**  Impreciso, depende de la escala (zoom).



Herramienta 1: El Método Gráfico (La Inspección Visual)

Concepto:

El método más simple consiste en graficar la función $y = f(x)$ y observar visualmente dónde cruza el eje X.

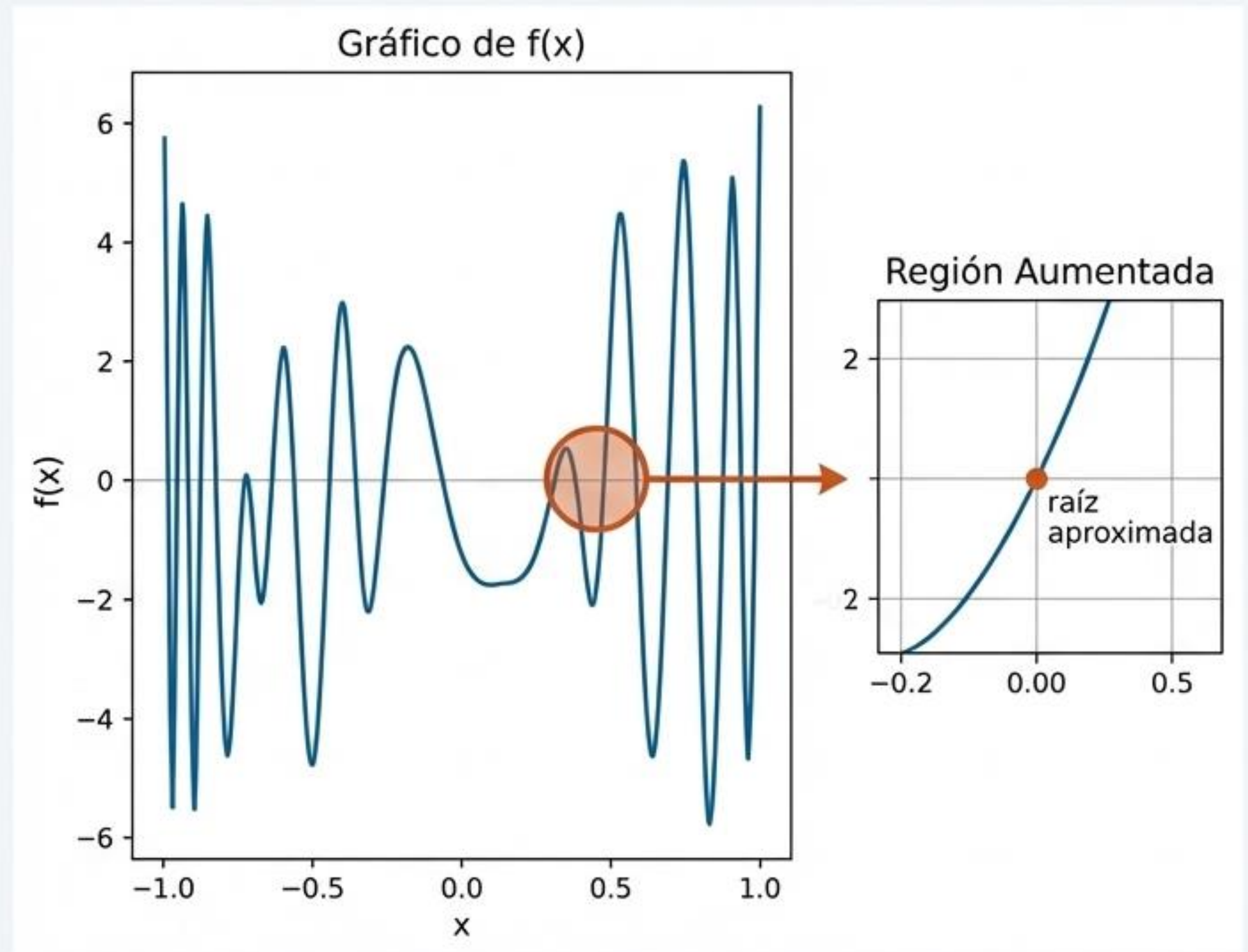
Proceso:

1. Generar un conjunto de puntos $(x, f(x))$ en un intervalo de interés.
2. Graficar estos puntos.
3. Identificar la(s) región(es) donde la gráfica intersecta el eje horizontal.

Utilidad:

- Proporciona una estimación inicial aproximada de la raíz.
- Ayuda a identificar la existencia y el número de raíces en un intervalo.
- Permite visualizar el comportamiento de la función.

Limitación: La precisión es baja y depende de la escala del gráfico.

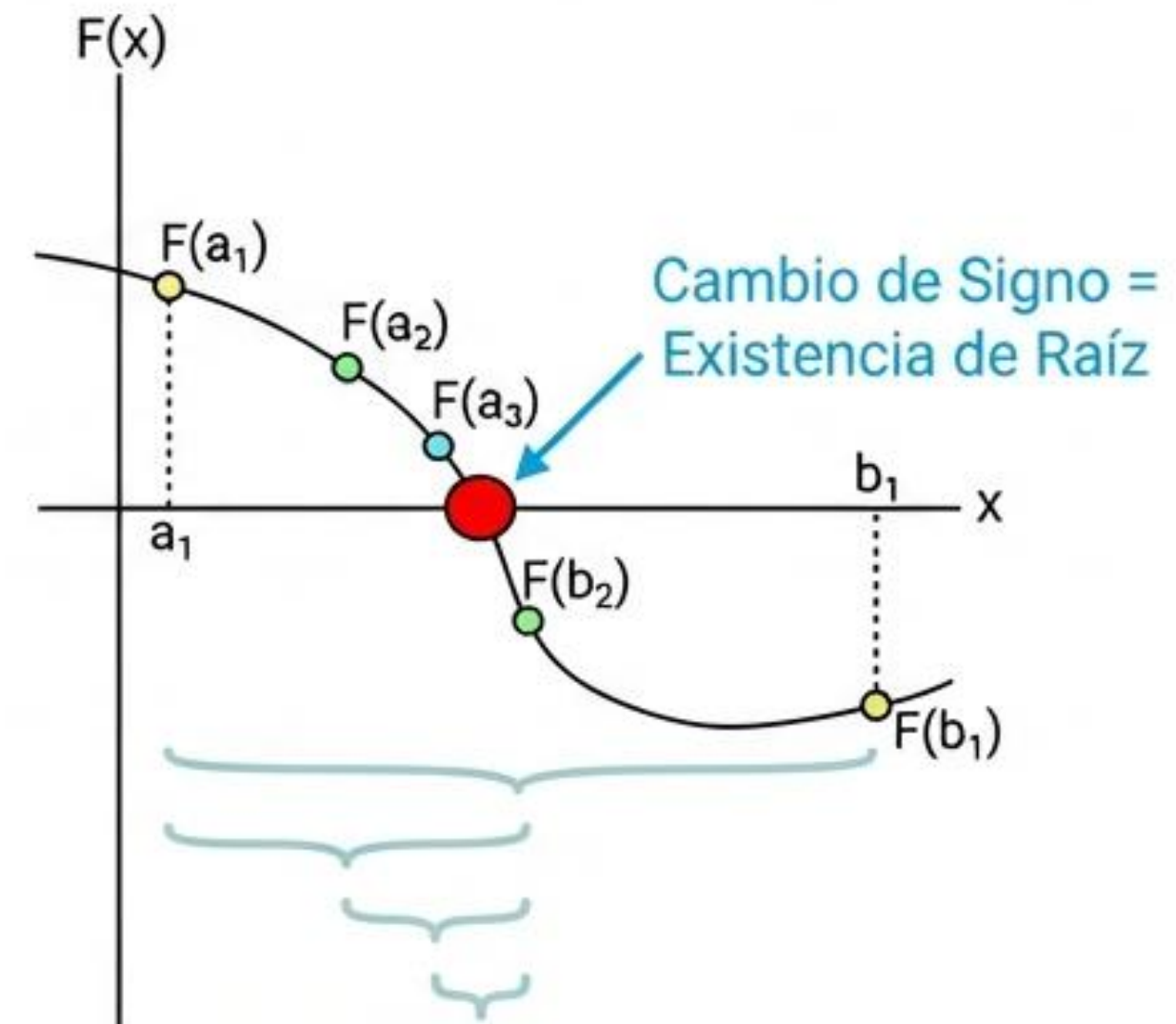


Teorema de Bolzano (Existencia)

Si f es continua en $[a, b]$ y $f(a) \cdot f(b) < 0$, entonces existe al menos una raíz $c \in (a, b)$.

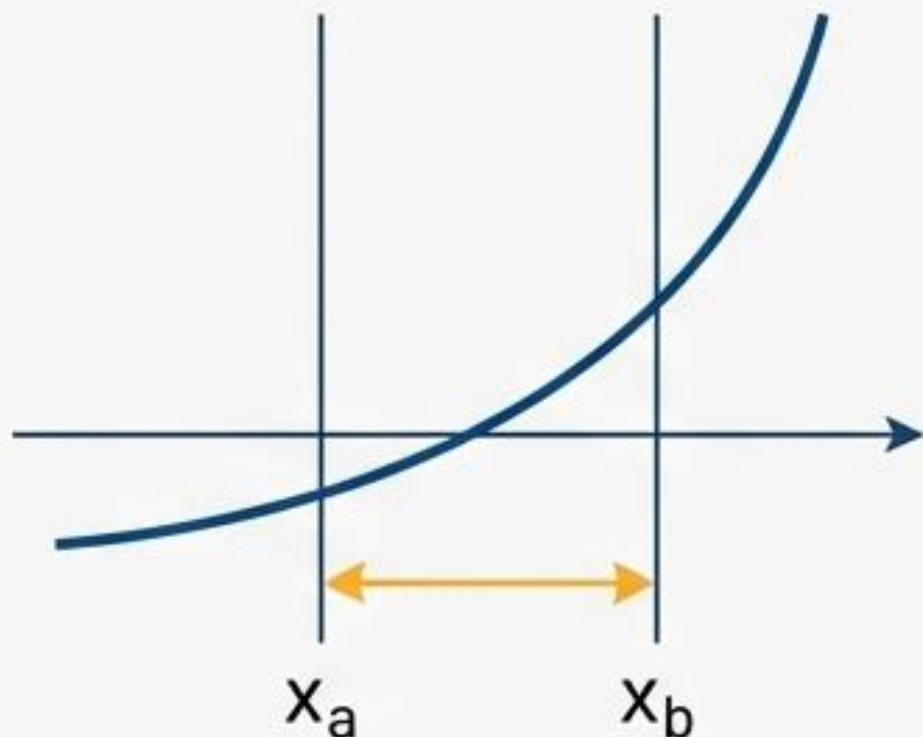


Bernard Bolzano (1781-1848)



Clasificación de Métodos Iterativos

Métodos Cerrados (Bracketing)

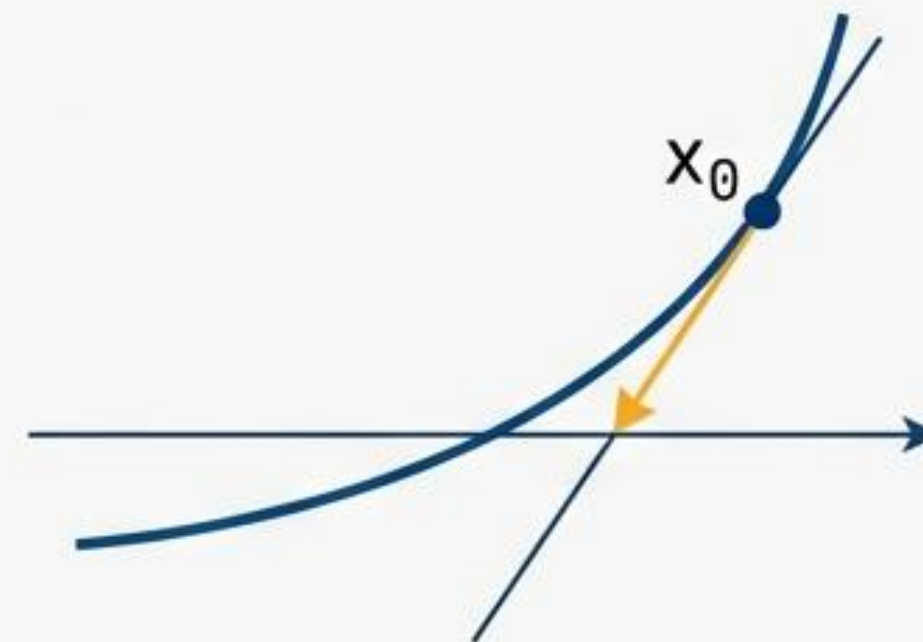


Concepto: Requieren dos valores iniciales que "encierran" la raíz.

Características: Convergencia garantizada, pero generalmente lenta.

Ejemplos: Método de Bisección, Regla Falsa.

Métodos Abiertos (Open Methods)



Concepto: Requieren un valor inicial (x_0) que no necesariamente encierra la raíz. Proyectan la solución.

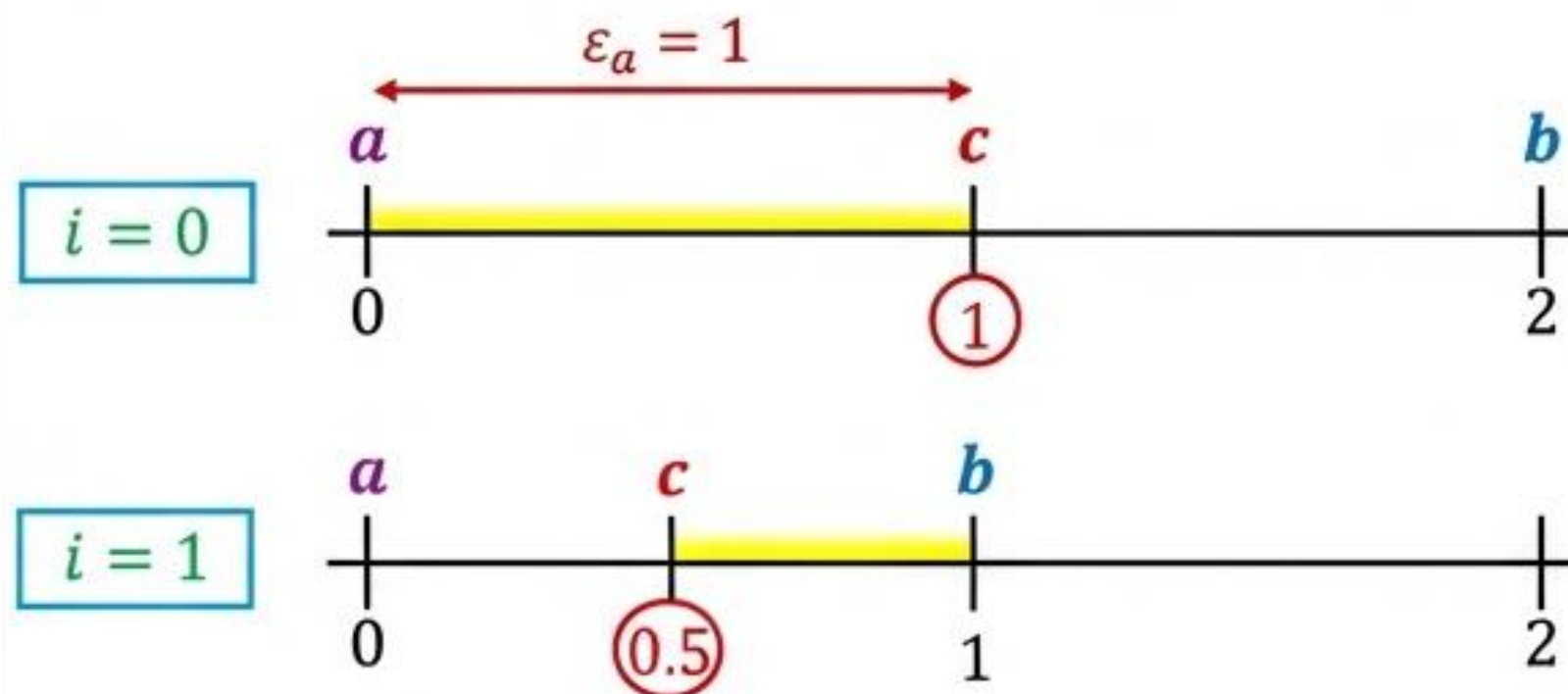
Características: Convergencia rápida (cuadrática), pero riesgo de divergencia.

Ejemplos: Newton-Raphson, Punto Fijo, Secante.

B. Métodos Cerrados: Bisección

Intuición Geométrica: Divide y Vencerás

1. Iniciar con intervalo $[a, b]$ donde los signos son opuestos.
2. Calcular punto medio $c = (a+b)/2$.
3. Desechar la mitad que no tiene cambio de signo.
4. Repetir.



$$\epsilon_a = 1 = \frac{2 - 0}{2^1}$$

Características: Convergencia siempre garantizada (robusto), pero lenta.

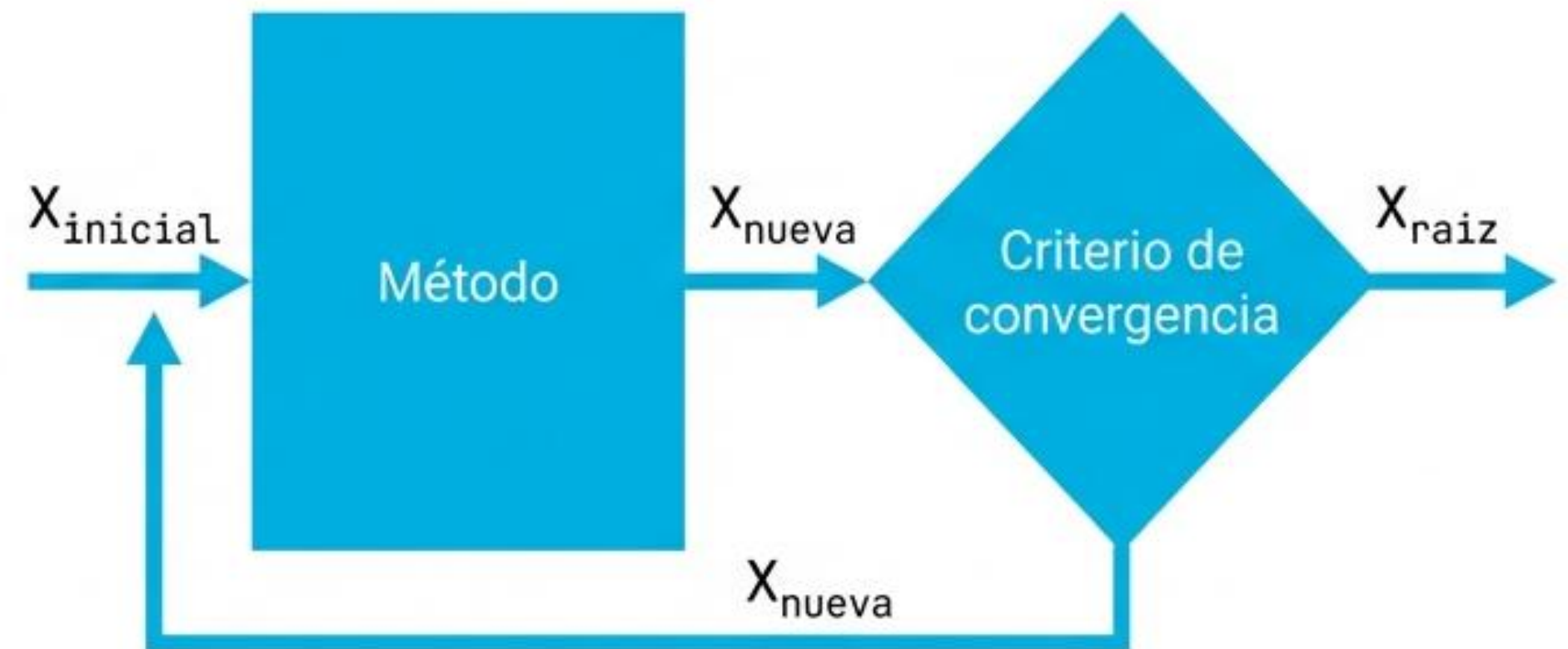
Algoritmo de Bisección

Pseudocódigo

```
Input: a, b, Tol
Check:  $f(a) \cdot f(b) < 0$ 
While  $(b - a) / 2 > \text{Tol}$ :
     $c = (a + b) / 2$ 
    If  $f(c) == 0$ : Stop

    If  $f(a) \cdot f(c) < 0$ :
         $b = c$  // Raíz a la izquierda
    Else:
         $a = c$  // Raíz a la derecha
Return c
```

Flujo Lógico del Método



El bucle 'While' y las condiciones 'If' se representan por la iteración entre 'Método' y el 'Criterio de convergencia'.

Análisis de Error en Bisección

Predicción "A Priori"

Cota de Error	Cota de Error
En la iteración n , el error absoluto es menor que la mitad del intervalo actual: $ x_{real} - x_n \leq \frac{b - a}{2^{n+1}}$	
Cálculo de Iteraciones	Cálculo de Iteraciones
Número de iteraciones necesarias (n) para una tolerancia (Tol): $n > \frac{\ln(b - a) - \ln(Tol)}{\ln 2}$	
Ejemplo	Ejemplo
Si intervalo = 1 y $Tol = 10^{-4}$: $\Rightarrow n > \frac{0 - (-9.21)}{0.69} \approx 13.28$ Resultado: Se requieren 14 iteraciones .	

C. Métodos Abiertos

Velocidad vs. Seguridad

Métodos Cerrados (Bisección)



Requieren un intervalo que encierre la raíz.

- ✓ Convergencia Garantizada
- ✗ Convergencia Lenta

VS

Métodos Abiertos (Newton, Punto Fijo)



Requieren solo un punto inicial x_0 .

- ✓ Convergencia Rápida (cuando funciona)
- ✗ Riesgo de Divergencia

Método del Punto Fijo

Transformación $x = g(x)$

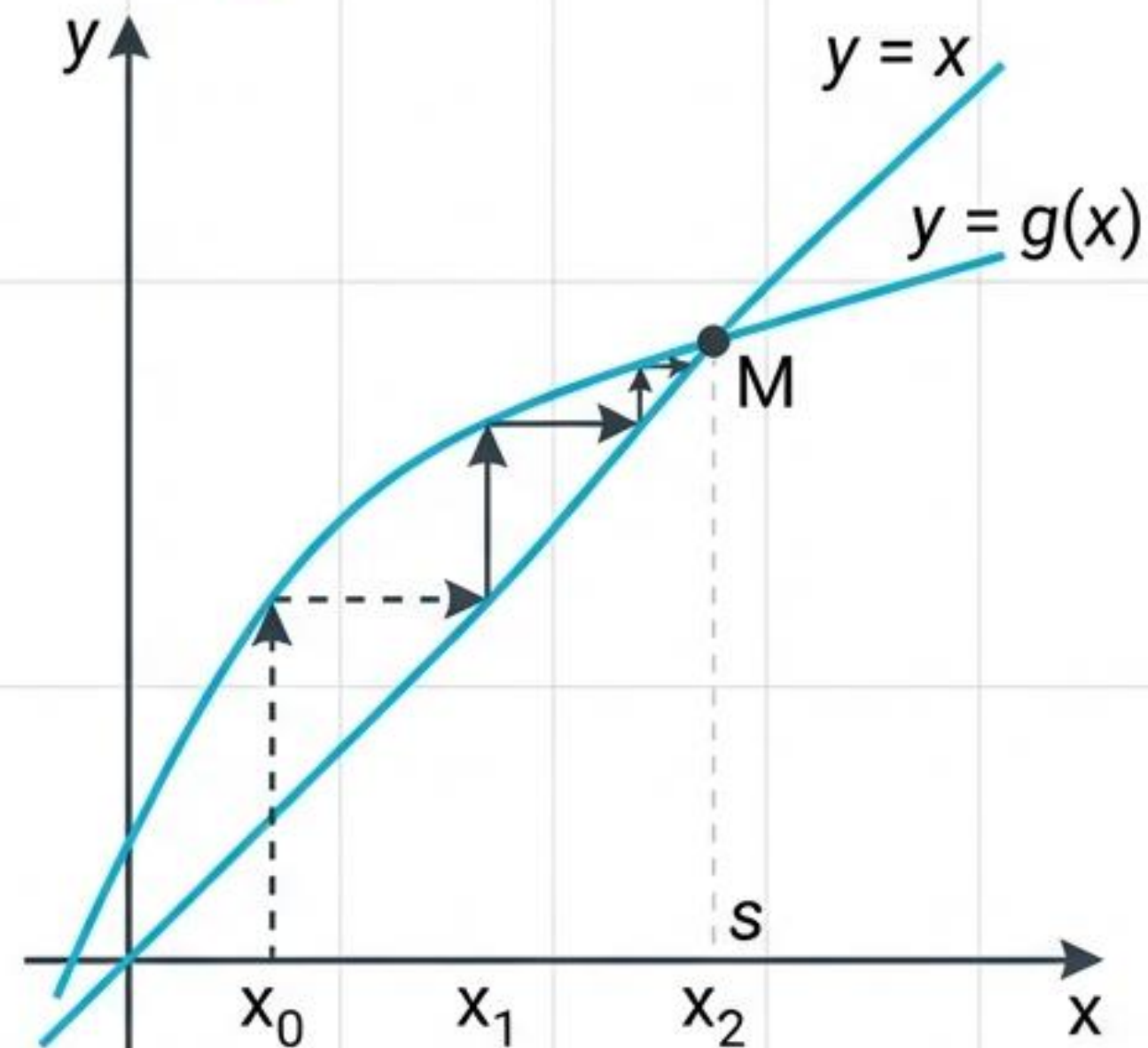
Algoritmo Iterativo:

- Transformar $f(x) = 0$ en $x = g(x)$.

$$x_{i+1} = g(x_i)$$

- Se busca la intersección de $y = g(x)$ con $y = x$.

Convergencia en Escalera

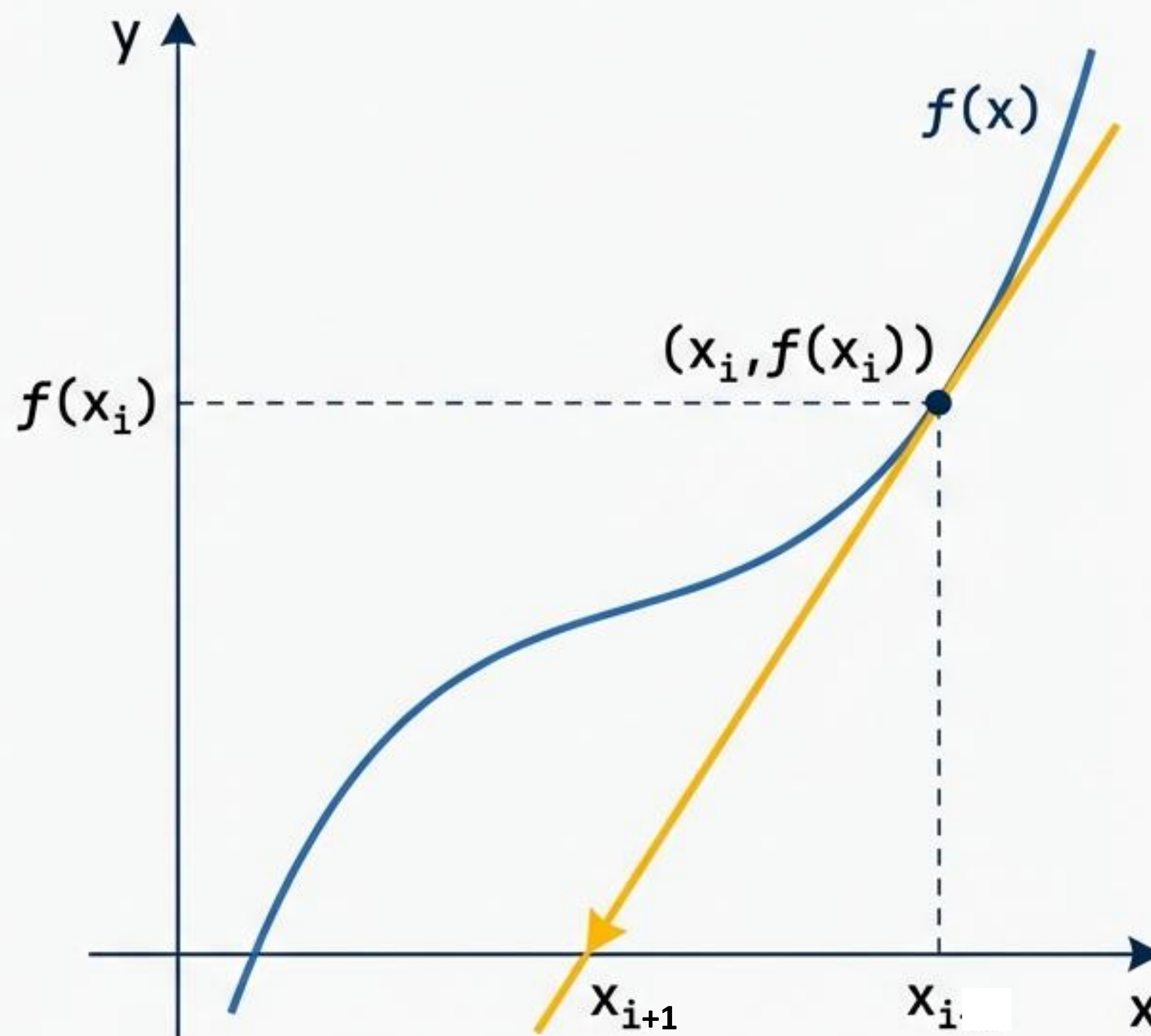


Métodos Abiertos: Newton-Raphson

Concepto: Utiliza la pendiente (derivada) de la función en un punto x_i para proyectar una línea tangente hacia el eje x , obteniendo una mejor estimación x_{i+1} .

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Ventaja Clave: Convergencia Cuadrática. El error es aproximadamente proporcional al cuadrado del error anterior, doblando la precisión en cada paso iterativo.

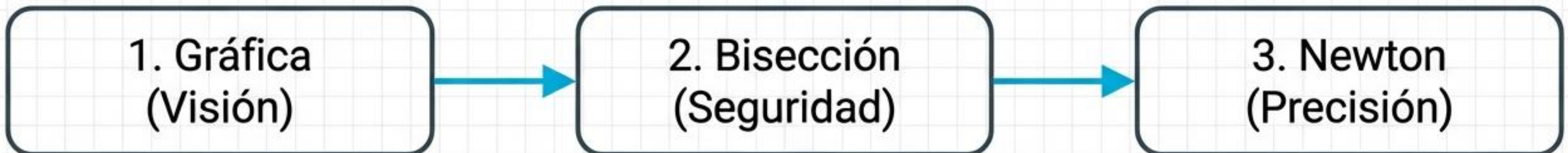


Resumen Comparativo

¿Qué herramienta usar?

Método	Tipo	Ventaja	Desventaja
Bisección	Cerrado	Garantizado (Bolzano)	Lento
Punto Fijo	Abierto	Flexible	Requiere $ g' < 1$
Newton	Abierto	Muy Rápido (Cuadrático)	Requiere derivada f'

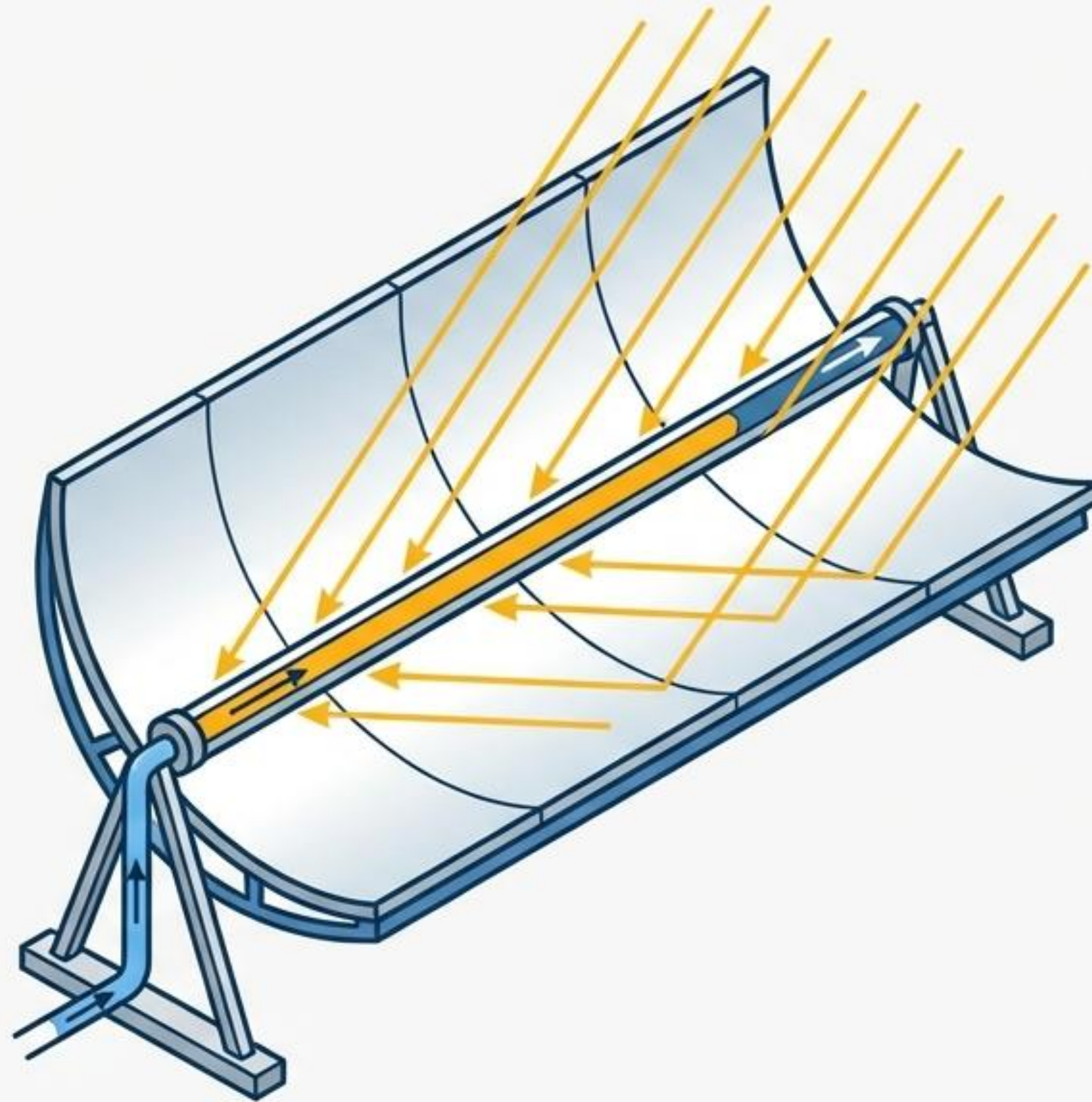
Flujo de Trabajo Recomendado (Ingeniería)



Aplicación en Ingeniería de Energía

Escenario: Diseño y control de colectores solares parabólicos.

El Desafío: Para maximizar la captura de energía, se debe determinar la temperatura de operación o el ángulo de incidencia donde la eficiencia es óptima.



Contexto Físico: La relación entre la radiación neta absorbida y la pérdida de calor por convección y radiación es altamente no lineal.

Modelado Matemático: Ecuación de Stefan-Boltzmann

$$f(T) = \varepsilon\sigma(T^4 - T_{\text{amb}}^4) - h(T - T_{\text{amb}}) = 0$$

Balance de Energía: Radiación Neta = Pérdida de Calor + Energía Almacenada

σ : Constante de Stefan-Boltzmann ($5.67 \times 10^{-8} \text{ W/m}^2\text{K}^4$)

ε : Emisividad de la superficie

h : Coeficiente de transferencia de calor por convección

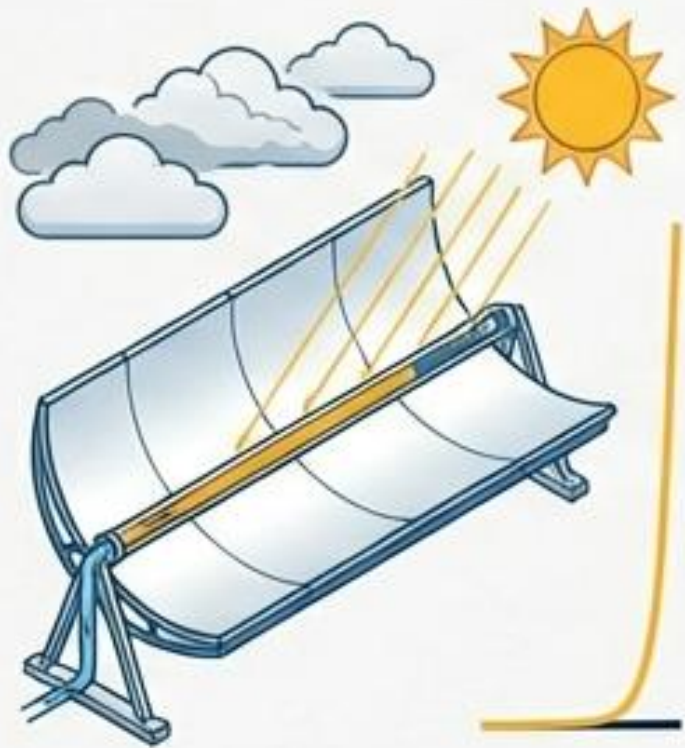
T : Temperatura de la superficie (Incógnita)

Observación: El término T^4 hace imposible despejar T explícitamente usando álgebra elemental, requiriendo métodos numéricos.

Selección del Algoritmo: ¿Por qué Newton-Raphson?

Velocidad vs. Seguridad

En sistemas de control en tiempo real (ajuste de espejos solares ante nubes pasajeras), la velocidad de cálculo es crítica.



- Newton-Raphson ofrece una convergencia mucho más rápida que Bisección, permitiendo ajustes casi instantáneos.

Implementación

- Se requiere calcular la derivada $f'(T)$ analíticamente una sola vez:

$$f'(T) = 4\epsilon\sigma T^3 - h$$

- Validación del Error: La convergencia se verifica controlando el error relativo:

$$\left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100 < \text{Tolerancia}$$

Actividad Práctica: "El Copiloto Digital"



Contexto

El ingeniero moderno no solo escribe código; supervisa y valida soluciones generadas por herramientas avanzadas.

Consigna (20 minutos):

1. **Prompting:** Utiliza una IA (ChatGPT, Claude o Gemini) para generar un script en MATLAB.
2. **Tarea:** Resolver la ecuación $x^3 - 2x^2 - 5 = 0$ (o el modelo de energía) usando el método de Newton-Raphson.
3. **Restricción:** El código debe solicitar el **valor inicial** y la **tolerancia de error**.

Validación Crítica del Código

```
MATLAB +
1 % Newton-Raphson Method Implementation
2 max_iter = 100; % Maximum iterations
3 tol = 1e-6; % Tolerance
4 x0 = 2; % Initial guess
5 x = x0;
6 for i = 1:max_iter
7     f_val = x^3 - 2*x^2 - 5;
8     f_prime_val = 3*x^2 - 4*x;
9     if abs(f_prime_val) < 1e-9 % Check for singularity
10         error('Derivative is near zero. Method fails.');
```

Checklist de Ingeniería:

- Condición de Parada:**
¿Incluye un límite máximo de iteraciones (`max_iter`) para evitar bucles infinitos si no converge?
- Tolerancia:** ¿Se detiene cuando el error es menor al `tol` deseado?
- Singularidades:** ¿Maneja el caso donde la derivada es cero ($f'(x) \approx 0$)? (Riesgo de división por cero).

Lección: La IA genera la sintaxis, el ingeniero **garantiza** la lógica numérica.

Referencias Bibliográficas

1. Chapra, S. C., & Canale, R. P. Métodos Numéricos para Ingenieros. McGraw-Hill Education.
2. Burden, R. L., & Faires, J. D. Análisis Numérico. Cengage Learning.
3. Nakamura, S. Métodos Numéricos Aplicados con Software. Prentice Hall.
4. Kiusalaas, J. Numerical Methods in Engineering with MATLAB. Cambridge University Press.

